Метод опорных векторов. Часть 3

. 3

Ядро SVM

Сходство наблюдений

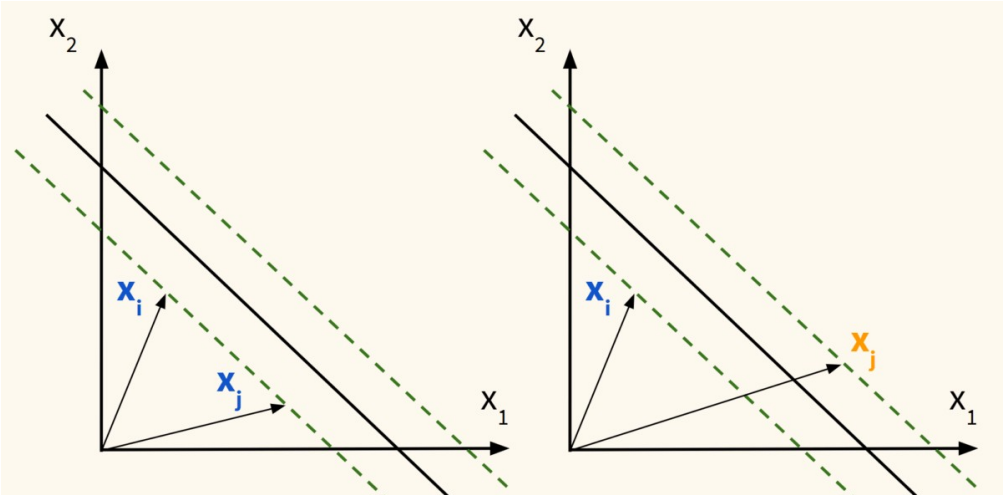
Вспомним, что задача SVM сводится к максимизации ширины коридора между наблюдениями через максимизацию двойственной функции

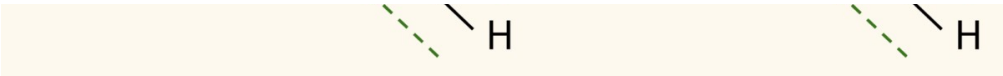
$$\max_{\boldsymbol{\alpha}} \mathcal{L}(\boldsymbol{\alpha}) = \max_{\boldsymbol{\alpha}} \left(\sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j \right),$$

Будем учитывать только те векторы, для которых $\alpha_i > 0$. Посмотрим на скалярное произведение с точки зрения сходства двух нормализованных векторов $\mathbf{x}_i$ и $\mathbf{x}_j$.

$$\cos\theta = \frac{\mathbf{x}_i \cdot \mathbf{x}_j}{||\mathbf{x}_i|| ||\mathbf{x}_j||}$$

В случае если векторы **не схожи**, их скалярное произведение близко к нулю, а значит второе слагаемое будет незначительно влиять на $\mathcal{L}$.

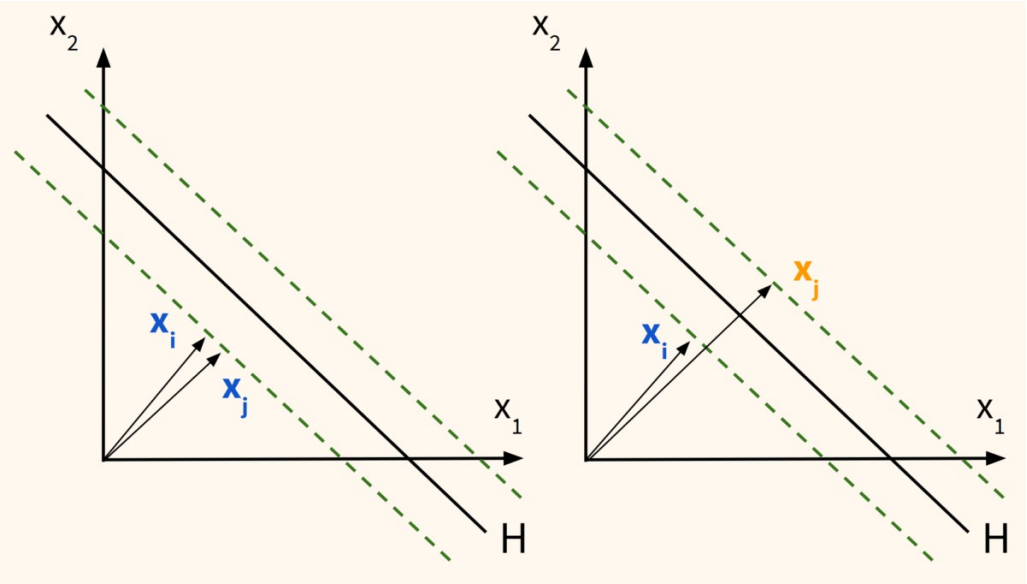




Если же векторы **схожи** и их скалярное произведение близко к единице, то возможны два варианта:

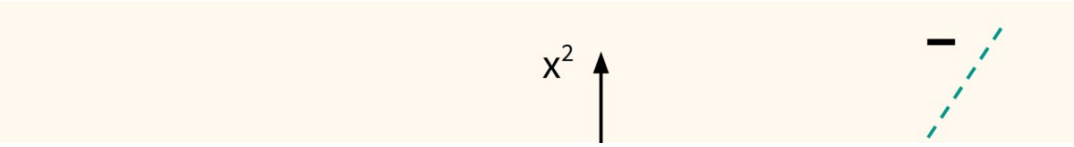
- *первай вариант*: объекты принадлежат к одному классу, например, для обоих $y_i = +1$ или $y_i = -1$; в этом случае их скалярное произведение будет положительным, а значит второе слагаемое в $\mathcal{L}$ будет вычитаться из первого и уменьшать целевую функцию, при оптимизации алгоритм будет отдавать меньшее предпочтение таким парам объектов;

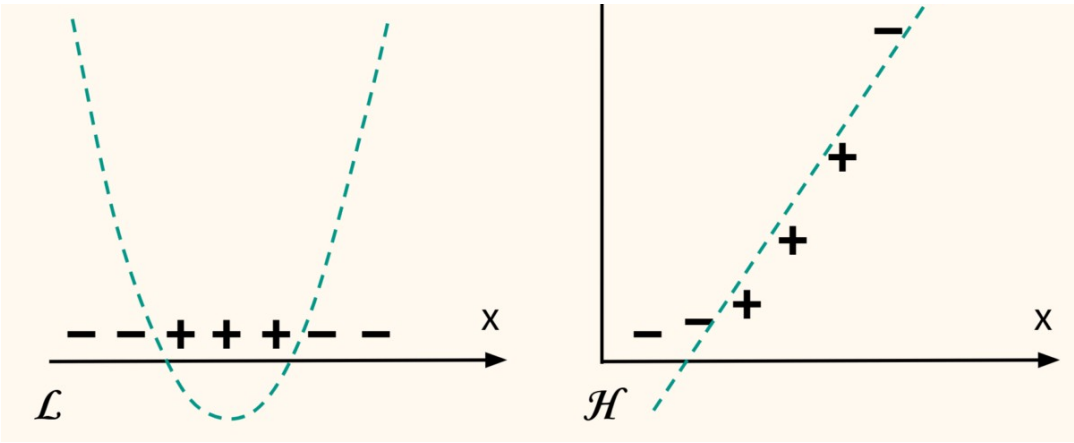
- *второй вариант*: объекты принадлежат к разным классам и их скалярное произведение всегда отрицательно; второе слагаемое прибавляется к первому и эти пары объектов способствуют максимизации $\mathcal{L}$ и, как следствие, ширины коридора между объектами.



Спрямление пространства

Рассмотрим данные с нелинейной разделяющей поверхностью в исходном пространстве признаков $\mathcal{L}$ (несложно увидеть, что где бы мы ни расположили *линейную* границу, количество ошибочно классифицированных объектов было бы слишком велико).





При этом эти же объекты могут быть линейно разделимы в другом пространстве (более высокой размерности) $\mathcal{H}$.

Одним из решений подобной задачи может быть добавление новых признаков, например, $x \mapsto \{x, x^2\}$ так, как мы это делали при изучении полиномиальной регрессии

$$h_\theta(x) = \theta_p x^p + \theta_{p-1} x^{p-1} + \ldots + \theta_1 x + \theta_0,$$

где, по сути, речь идет о **базисной функции** (basis function) $\varphi : \mathbb{R} \to \mathbb{R}^p$,

$$\varphi(x) = \begin{bmatrix} 1 \\ x \\ x^2 \\ \vdots \\ x^p \end{bmatrix}.$$

Полиномиальную модель, таким образом, можно записать как

$$h_\theta(x) = \sum_{i=0}^{p} \theta_p x^p = \theta^\top \varphi(x)$$

Другими словам, мы применили линейную модель h_θ к преобразованным признакам $\varphi(x)$. При этом преобразование признаков может быть затратным.

Ядро и kernel trick

Аналогичным образом мы можем применить метод опорных векторов к нелинейным зависимостям

$$\mathcal{L}(\boldsymbol{\alpha}) = \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j y_i y_j \varphi(\mathbf{x}_i)^\top \varphi(\mathbf{x}_j).$$

Таким образом мы добиваемся преобразования признаков $\varphi(x)$ и сохраняем скалярное произведение, которое, будучи мерой сходства двух объектов, способствует максимизации зазора.

Более того, нам не обязательно *предварительно* преобразовывать признаки. Предположим, что мы хотели бы сформировать следующие нелинейные признаки для единственного признака a

$$\varphi(a) = \begin{bmatrix} \sqrt{2}a \\ a^2 \\ 1 \end{bmatrix},$$

то есть масштабировать признак на величину $\sqrt{2}$, найти квадрат этого признака и добавить единицу. Теперь найдем скалярное произведение двух подобных объектов $\varphi(a)$ и $\varphi(b)$

$$\begin{aligned} \varphi(a) \cdot \varphi(b) &= (\sqrt{2}a, a^2, 1) \cdot (\sqrt{2}b, b^2, 1) \\ &= 2ab + a^2b^2 + 1 \\ &= (a \times b + 1)(a \times b + 1) \\ &= (a \times b + 1)^2 \end{aligned}$$

Другими словами, скалярное произведение преобразованных признаков $\varphi(a)$ и $\varphi(b)$ можно заменить на обычное произведение, сложение и возведение в степень.

То же самое справедливо и для векторов $\mathbf{a}$ и $\mathbf{b}$. Пусть отображение имеет вид

$$\varphi(\mathbf{a}) = \varphi\left(\begin{pmatrix} a_1 \\ a_2 \end{pmatrix}\right) = \begin{pmatrix} a_1^2 \\ \sqrt{2}a_1a_2 \\ a_2^2 \end{pmatrix}$$

Найдем скалярное произведение

$$
\begin{aligned}
\varphi(\mathbf{a})^\top \varphi(\mathbf{b}) &= \begin{pmatrix} a_1^2 \\ \sqrt{2}a_1a_2 \\ a_2^2 \end{pmatrix}^\top \begin{pmatrix} b_1^2 \\ \sqrt{2}b_1b_2 \\ b_2^2 \end{pmatrix} \\
&= a_1^2b_1^2 + 2a_1b_1a_2b_2 + a_2^2b_2^2 \\
&= (a_1b_1 + a_2b_2)^2 \\
&= \left(\begin{pmatrix} a_1 \\ a_2 \end{pmatrix}^\top \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} \right)^2 \\
&= (\mathbf{a}^\top \mathbf{b})^2
\end{aligned}
$$

Заметим, что функция $K(\mathbf{x}_i, \mathbf{x}_j) = \varphi(\mathbf{x}_i)^\top \varphi(\mathbf{x}_j)$ называется **ядром** (kernel function), а возможность использовать скалярное произведение исходных признаков без преобразования принято называть **kernel trick**.

Линейное ядро

Повторимся, что $\mathbf{x}_i^\top \mathbf{x}_j$ называют **линейным ядром** (linear kernel).

$$K(\mathbf{x}_i, \mathbf{x}_j)_{\text{linear}} = \mathbf{x}_i^\top \mathbf{x}_j$$

Полиномиальное ядро

Кроме того, выше мы уже рассмотрели **полиномиальное ядро** (polynomial kernel)

$$K(\mathbf{x}_i, \mathbf{x}_j)_{\text{poly}} = (\gamma \cdot \mathbf{x}_i^\top \mathbf{x}_j + c)^d,$$

где γ контролирует влияние отдельного объекта на решающую границу.

Ядро радиальной базисной функции

Рассмотрим **ядро радиальной базисной функции** (Radial Basis Function kernel, RBF) или **гауссовское ядро** (Gaussian Kernel).

$$K(\mathbf{x}_i, \mathbf{x}_j)_{\text{rbf}} = \exp\left(-\frac{||\mathbf{x}_i - \mathbf{x}_j||^2}{2\sigma^2}\right) = \exp\left(-\gamma||\mathbf{x}_i - \mathbf{x}_j||^2\right),$$

где $\gamma = \frac{1}{2\sigma^2}$. Мы опускаем нормализующую константу $\left(\frac{1}{\sigma\sqrt{2\pi}}\right)$, поскольку речь не идет о вероятностном распределении.

Приведем несколько простых примеров одномерных данных:

- пусть $a = 2$ и $b = 3$, а $\gamma = 1$, тогда $\exp(-(2-3)^2) \approx 0{,}37$;
- если же $a = 2$ и $b = 4$, то $\exp(-(2-4)^2) \approx 0{,}18$;
- наконец пусть $a = 2, b = 3$, но $\gamma = 2$, в этом случае $\exp(-2(2-3)^2) \approx 0{,}14$.

Другими словами, чем больше расстояние между объектами или чем больше γ, тем меньше значение функции ядра.

Кроме того, покажем, что ядро радиальной базисной функции является скалярным произведением $\varphi(\mathbf{x}_i)^\top \varphi(\mathbf{x}_j)$ в бесконечномерном пространстве.

Рассмотрим полиномиальное ядро, в котором $c = 0$ и данные имеют только одно измерение

$$(ab)^d = a^d b^d$$

Теперь возьмем сумму всех возможных степеней полиномиальных ядер вплоть до бесконечности и заметим, что речь идет о скалярном произведении бесконечномерных векторов

$$(a^1 b^1 + a^2 b^2 + \ldots + a^\infty b^\infty) =$$

$$(a, a^2, \ldots, a^\infty) \cdot (b, b^2, \ldots, b^\infty)$$

Обратимся к формуле ядра RBF

$$\exp(-\gamma(a-b)^2) = \exp(-\gamma(a^2 + b^2 - 2ab)) = \exp(-\gamma(a^2 + b^2)) \exp(\gamma 2ab)$$

Положим $\gamma = \frac{1}{2}$, тогда

$$\exp\left(-\frac{1}{2}(a^2+b^2)\right)\exp\left(\frac{2}{2}ab\right)=\exp\left(-\frac{1}{2}(a^2+b^2)\right)\exp(ab)$$

Разложим $\exp(ab)$ в ряд Маклорена

$$\exp(ab)=1+\frac{1}{1!}ab+\frac{1}{2!}(ab)^2+\ldots+\frac{1}{\infty!}(ab)^\infty$$

Заметим, что разложение $\exp(ab)$ аналогично

$$\exp(ab)=\left(a^0b^0+\frac{1}{1!}a^1b^1+\frac{1}{2!}a^2b^2+\ldots+\frac{1}{\infty!}a^\infty b^\infty\right)=$$

$$\left(1,\sqrt{\frac{1}{1!}}a,\sqrt{\frac{1}{2!}}a^2,\ldots,\sqrt{\frac{1}{\infty!}}a^\infty)\cdot(1,\sqrt{\frac{1}{1!}}b,\sqrt{\frac{1}{2!}}b^2,\ldots,\sqrt{\frac{1}{\infty!}}b^\infty\right)$$

Положим $s=\sqrt{\exp\left(-\frac{1}{2}(a^2+b^2)\right)}$. Тогда,

$$s\left[\left(1,\sqrt{\frac{1}{1!}}a,\sqrt{\frac{1}{2!}}a^2,\ldots,\sqrt{\frac{1}{\infty!}}a^\infty)\cdot(1,\sqrt{\frac{1}{1!}}b,\sqrt{\frac{1}{2!}}b^2,\ldots,\sqrt{\frac{1}{\infty!}}b^\infty\right)\right]=$$

$$\left(s,s\sqrt{\frac{1}{1!}}a,s\sqrt{\frac{1}{2!}}a^2,\ldots,s\sqrt{\frac{1}{\infty!}}a^\infty)\cdot(s,s\sqrt{\frac{1}{1!}}b,s\sqrt{\frac{1}{2!}}b^2,\ldots,s\sqrt{\frac{1}{\infty!}}b^\infty\right)$$

Другими словами, значение радиальной базисной функции из примера выше

$$K_{\text{rbf}}(2,3)=\exp(-(2-3)^2)\approx 0{,}37$$

показывает степень сходства этих объектов в бесконечномерном пространстве.

Мягкий зазор и ядро

Важно понимать, что ядро позволяет сделать данные линейно разделимыми в другом пространстве (более высокой размерности). Параметр C позволяет создать эффективный классификатор даже если и в этом пространстве данные не до конца линейно разделимы (то есть имеют выбросы). У этих подходов разные задачи.

Параметры и прогноз

$$b^* = \frac{1}{S}\sum_{i=1}^{S}(y_i - \sum_{i=1}^{S}\alpha_i y_i K(\mathbf{x}_i, \mathbf{x}_j))$$

Опять же мы можем рассчитать b^* на основе одного опорного вектора.

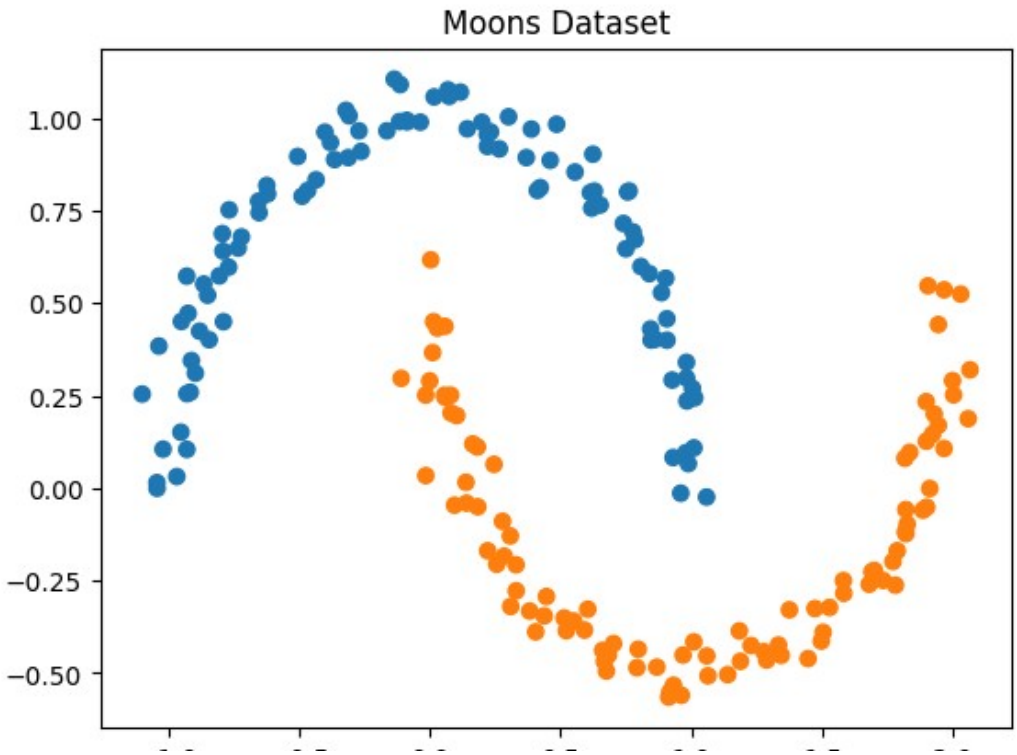
$$\hat{y} = sgn\left(\sum_{i=1}^{S}\alpha_i y_i K(\mathbf{x}_i, \mathbf{x}') + b^*\right)$$

Код на Питоне

Вначале создадим линейно неразделимые данные.

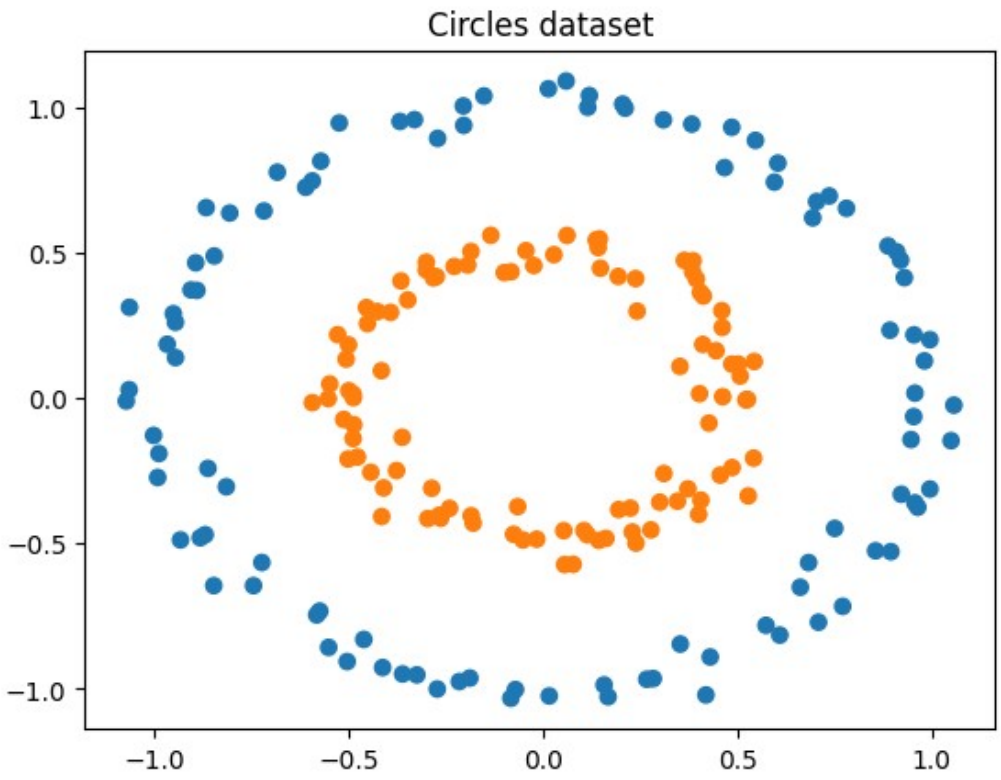
Продолжим в том же блокноте⧉

```
X_moons, y_moons = make_moons(n_samples=200, noise=.05, random_state=42)
y_moons = np.where(y_moons <= 0, -1, 1)
plt.scatter(X_moons[:, 0], X_moons[:, 1],
            c=y_moons, cmap=colors.ListedColormap(clrs))
plt.title('Moons Dataset')
plt.show()
```



−1.0 −0.5 0.0 0.5 1.0 1.5 2.0

```
X_circles, y_circles = make_circles(n_samples=200, factor=.5, noise=.05,
y_circles = np.where(y_circles <= 0, -1, 1)
plt.scatter(X_circles[:, 0], X_circles[:, 1],
            c=y_circles, cmap=colors.ListedColormap(clrs))
plt.title('Circles dataset')
plt.show()
```



Напишем класс KernelSVM.

```
class KernelSVM:

  def __init__(self, C=3, kernel='poly', degree=4, gamma=None, coef=2):
    self.C = C
    self.kernel = kernel
    self.degree = degree
    self.gamma = gamma
    self.coef = coef
    self.lagr_multipliers = None
    self.support_vectors = None
    self.support_vector_labels = None
    self.intercept = None

  def _kernel(self, x1, x2):
    if self.kernel == 'linear':
      return np.dot(x1, x2)
    elif self.kernel == 'poly':
```

```
      return (self.gamma * np.dot(x1, x2) + self.coef) ** self.degree
    elif self.kernel == 'rbf':
      return np.exp(-self.gamma * np.linalg.norm(x1 - x2) ** 2)

  def fit(self, X, y):
    n_samples, n_features = np.shape(X)
    if self.gamma is None:
      self.gamma = 1 / n_features

    K = np.zeros((n_samples, n_samples))
    for i in range(n_samples):
      for j in range(n_samples):
        K[i, j] = self._kernel(X[i], X[j])

    P = cvxopt.matrix(np.outer(y, y) * K, tc='d')
    q = cvxopt.matrix(np.ones(n_samples) * -1)
    A = cvxopt.matrix(y, (1, n_samples), tc='d')
    b = cvxopt.matrix(0.0)

    if not self.C:
      G = cvxopt.matrix(np.identity(n_samples) * -1)
      h = cvxopt.matrix(np.zeros(n_samples))
    else:
      G_max = np.identity(n_samples) * -1
      G_min = np.identity(n_samples)
      G = cvxopt.matrix(np.vstack((G_max, G_min)))
      h_max = cvxopt.matrix(np.zeros(n_samples))
      h_min = cvxopt.matrix(np.ones(n_samples) * self.C)
      h = cvxopt.matrix(np.vstack((h_max, h_min)))

    solution = cvxopt.solvers.qp(P, q, G, h, A, b)

    alphas = np.ravel(solution['x'])

    idx = alphas > 1e-5
    self.support_alphas = alphas[idx]
    self.support_vectors = X[idx]
    self.support_vector_labels = y[idx]

    # рассчитаем b на основе первого опорного вектора self.support_vector
    temp_sum = 0
    for i in range(len(self.support_alphas)):
      temp_sum += self.support_alphas[i] * self.support_vector_labels[
          i] * self._kernel(self.support_vectors[i], self.support_vectors
    self.intercept = self.support_vector_labels[0] - temp_sum

    return self

  def predict(self, X):
    scores, y_pred = [], []
```

```
    for sample in X:
      prediction = 0
      for i in range(len(self.support_alphas)):
        prediction += self.support_alphas[i] * self.support_vector_labels
            i] * self._kernel(self.support_vectors[i], sample)
      prediction += self.intercept

      scores.append(prediction)
      y_pred.append(np.sign(prediction))

    return np.array(y_pred), np.array(scores)
```

Применим полиномиальное ядро степени три к набору данных moons.

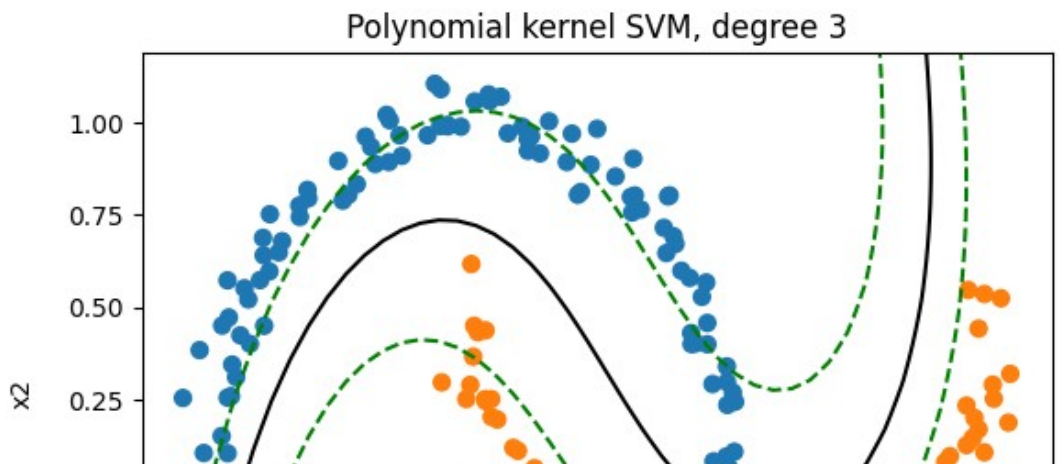
```
poly_kernel_model = KernelSVM(kernel='poly', degree=3).fit(X_moons, y_moo
```

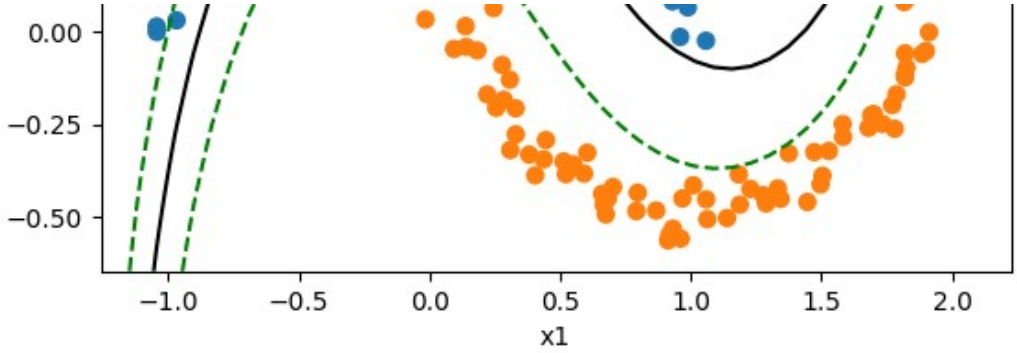
```
accuracy_score(y_moons, poly_kernel_model.predict(X_moons)[0])
```

```
1.0
```

Посмотрим на разделяющую границу на графике.

```
plt.scatter(X_moons[:, 0], X_moons[:, 1],
            c=y_moons, s=40, cmap=colors.ListedColormap(clrs))
ax = plt.gca()
xlim = ax.get_xlim()
ylim = ax.get_ylim()
xx = np.linspace(xlim[0], xlim[1], 50)
yy = np.linspace(ylim[0], ylim[1], 50)
YY, XX = np.meshgrid(yy, xx)
xy = np.vstack([XX.ravel(), YY.ravel()]).T
# найдем нелинейную разделяющую границу
Z = poly_kernel_model.predict(xy)[1].reshape(XX.shape)
ax.contour(XX, YY, Z, levels=[-1, 0, 1], colors=['g','k','g'], linestyles
plt.xlabel('x1')
plt.ylabel('x2')
plt.title('Polynomial kernel SVM, degree 3')
plt.show()
```





Поясним, что линейной границе в $\mathcal{H}$ соответствует нелинейная граница в $\mathcal{L}$.

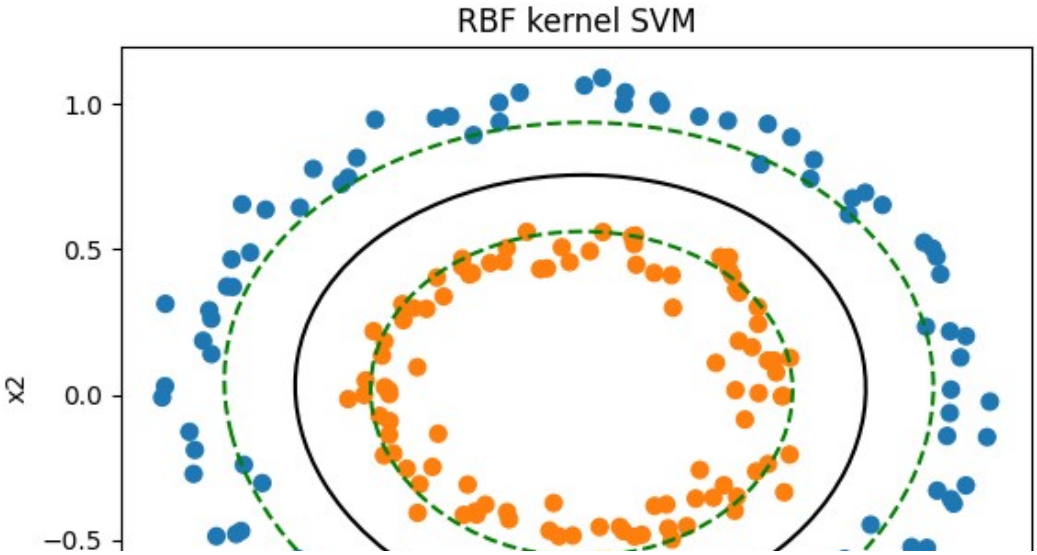
Теперь применим ядро радиальной базисной функции к датасету circles.

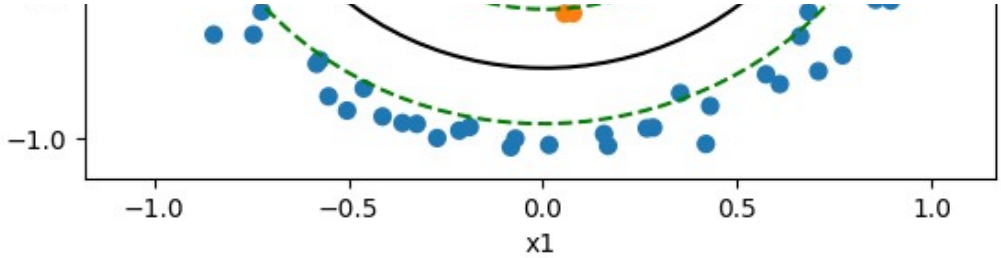
```
rbf_kernel_model = KernelSVM(kernel='rbf').fit(X_circles, y_circles)
```

```
accuracy_score(y_circles, rbf_kernel_model.predict(X_circles)[0])
```

```
1.0
```

```
plt.scatter(X_circles[:, 0], X_circles[:, 1],
            c=y_circles, s=40, cmap=colors.ListedColormap(clrs))
ax = plt.gca()
xlim = ax.get_xlim()
ylim = ax.get_ylim()
xx = np.linspace(xlim[0], xlim[1], 50)
yy = np.linspace(ylim[0], ylim[1], 50)
YY, XX = np.meshgrid(yy, xx)
xy = np.vstack([XX.ravel(), YY.ravel()]).T
Z = rbf_kernel_model.predict(xy)[1].reshape(XX.shape)
ax.contour(XX, YY, Z, levels=[-1, 0, 1], colors=['g','k','g'], linestyles
plt.xlabel('x1')
plt.ylabel('x2')
plt.title('RBF kernel SVM')
plt.show()
```





Список литературы

- Christopher J.C. Burges, A Tutorial on Support Vector Machines for Pattern Recognition
- Chuong B. Do, Convex Optimization Overview
- Andrew Ng, CS229 Lecture notes. Part V. SVM
- Воронцов К.В., Лекции по методу опорных векторов
- Shawe-Taylor J., Cristianini N., Kernel Methods for Pattern Analysis, Cambridge University Press, 2004